```cpp
LL dfs(LL a, LL b, LL c)
{
    if(a <= 0 || b <= 0 || c <= 0) return 1;
    if(a > 20 || b > 20 || c > 20) return dfs(20, 20, 20);

    if(f[a][b][c]) return f[a][b][c];

    if(a < b && b < c) return f[a][b][c] = dfs(a, b, c - 1) + dfs(a, b - 1, c - 1) - dfs(a, b - 1, c);
    else return f[a][b][c] = dfs(a - 1, b, c) + dfs(a - 1, b - 1, c) + dfs(a - 1, b, c - 1) - dfs(a - 1, b - 1, c - 1);
}

int main()
{
    while(cin >> a >> b >> c)
    {
        // 多组测试数据：不需要清空
        if(a == -1 && b == -1 && c == -1) break;

        printf("w(%lld, %lld, %lld) = %lld\n", a, b, c, dfs(a, b, c));
    }

    return 0;
}
```

1.4.2 天下第一

题目来源： 洛谷

题目链接： P5635【CSGRound1】天下第一

难度系数： ★★

【题目描述】

游戏是这样的：

给定两个数 x, y，与一个模数 p。

cbw 拥有数 x，zhouwc 拥有数 y。

第一个回合：$x \leftarrow (x + y) \bmod p$

第二个回合：$y \leftarrow (x + y) \bmod p$

第三个回合：$x \leftarrow (x + y) \bmod p$

第四个回合：$y \leftarrow (x + y) \bmod p$

> 以此类推....
>
> 如果 x 先到 0 ，则 cbw 胜利。如果 y 先到 0 ，则 zhouwc 胜利。如果 x,y 都不能到 0 ，则为平局。
>
> cbw 为了捍卫自己主席的尊严，想要提前知道游戏的结果，并且可以趁机动点手脚，所以他希望你来告诉他结果。

【输入描述】

> 有多组数据。
>
> 第一行： T 和 p 表示一共有 T 组数据且模数都为 p 。
>
> 以下 T 行，每行两个数 x,y 。
>
> $1 \leq T \leq 200$
>
> $1 \leq x,y,p \leq 10000$

【输出描述】

> 共 T 行
>
> 1 表示 cbw 获胜， 2 表示 zhouwc 获胜， `error` 表示平局。

【示例一】

> 输入：
>
> 1 10
>
> 1 3
>
> 输出：
>
> error

【示例二】

> 输入：
>
> 1 10
>
> 4 5
>
> 输出：
>
> 1

【解法】

用递归模拟整个游戏过程： $dfs(x,y)$ 的结果可以由 $dfs((x+y)\%p,(x+y+y)\%p)$ 得到。

因为测试数据是多组的，并且模数都是 p ，再加上递归的过程中会递归的相同的问题，所以可以把递归改写成记忆化搜索。其中：

- $f[x][y] = 1$，表示 cbw 赢；
- $f[x][y] = 2$，表示 $zhouwc$ 赢；
- $f[x][y] = 3$ 表示这个位置已经被访问过，如果没被修改成 1 或者 2 ，那就表明平局。

注意事项：

- 这道题的数据范围很大，用 int 类型创建二维数组空间会溢出。但是我们的最终结果仅有三种情况，所以可以用 $char$ 类型来存储最终结果，节省空间。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e4 + 10;

int x, y, p;
char f[N][N]; // 备忘录

char dfs(int x, int y)
{
    if(f[x][y]) return f[x][y]; // 剪枝

    f[x][y] = '3'; // 这个状态已经访问过了，之后再遇到时，表示平局

    if(x == 0) return f[x][y] = '1';
    if(y == 0) return f[x][y] = '2';
    return f[x][y] = dfs((x + y) % p, (x + y + y) % p);
}

int main()
{
    int T; cin >> T >> p;

    while(T--)
    {
        cin >> x >> y;
        char ret = dfs(x, y);
        if(ret == '1') cout << 1 << endl;
        else if(ret == '2') cout << 2 << endl;
        else cout << "error" << endl;
    }
```

```
33
34        return 0;
35    }
```

1.4.3 滑雪

题目来源： 洛谷

题目链接： P1434 [SHOI2002] 滑雪

难度系数： ★★

【题目描述】

Michael 喜欢滑雪。这并不奇怪，因为滑雪的确很刺激。可是为了获得速度，滑的区域必须向下倾斜，而且当你滑到坡底，你不得不再次走上坡或者等待升降机来载你。Michael 想知道在一个区域中最长的滑坡。区域由一个二维数组给出。数组的每个数字代表点的高度。下面是一个例子：

1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9

一个人可以从某个点滑向上下左右相邻四个点之一，当且仅当高度会减小。在上面的例子中，一条可行的滑坡为 $24-17-16-1$（从 24 开始，在 1 结束）。当然 $25-24-23-\ldots-3-2-1$ 更长。事实上，这是最长的一条。

【输入描述】

输入的第一行为表示区域的二维数组的行数 R 和列数 C。下面是 R 行，每行有 C 个数，代表高度(两个数字之间用 1 个空格间隔)。

对于 100% 的数据，$1 \le R, C \le 100$。

【输出描述】

输出区域中最长滑坡的长度。

【示例一】

输入：

```
5 5
1 2 3 4 5
16 17 18 19 6
```

> 15 24 25 20 7
>
> 14 23 22 21 8
>
> 13 12 11 10 9
>
> 输出：
>
> 25

【解法】

暴力枚举：遍历整个矩阵，看看以当前位置为起点，最远能滑行多远的距离。在所有情况里面，取最大值即可。

如何求出以 $[i,j]$ 为起点的最大距离？

- 从 $[i,j]$ 位置上下左右瞅一瞅，如果能滑过去，就看看以下一个位置为起点，最远能滑行多远的距离；
- 找出四个方向上的最远距离，然后 $+1$ 。

因为出现相同子问题，所以可以用 dfs 来解决。又因为在搜索的过程中会遇到一模一样的问题，因此可以把递归改成记忆化搜索的方式。

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 110;

int n, m;
int a[N][N];
int f[N][N]; // 备忘录

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

int dfs(int i, int j)
{
    if(f[i][j]) return f[i][j];

    int len = 1;
    // 上下左右四个方向搜
    for(int k = 0; k < 4; k++)
```

```
21      {
22          int x = i + dx[k], y = j + dy[k];
23          if(x < 1 || x > n || y < 1 || y > m) continue;
24          if(a[i][j] <= a[x][y]) continue;
25
26          len = max(dfs(x, y) + 1, len);
27      }
28      return f[i][j] = len;
29  }
30
31  int main()
32  {
33      cin >> n >> m;
34      for(int i = 1; i <= n; i++)
35          for(int j = 1; j <= m; j++)
36              cin >> a[i][j];
37
38      int ret = 1;
39      for(int i = 1; i <= n; i++)
40          for(int j = 1; j <= m; j++)
41              ret = max(ret, dfs(i, j));
42
43      cout << ret << endl;
44
45      return 0;
46  }
```

2. 宽度优先搜索 - BFS

宽度优先搜索的过程中，每次都会从当前点向外扩展一层，所以会具有一个最短路的特性。因此，宽搜不仅能搜到所有的状态，而且还能找出起始状态距离某个状态的最小步数。

但是，前提条件是每次扩展的代价都是 1 ，或者都是相同的数。宽搜常常被用于解决边权为 1 的最短路问题。

2.1 BFS

2.1.1 马的遍历

> **题目来源：** 洛谷
>
> **题目链接：** P1443 马的遍历
>
> **难度系数：** ★★

【题目描述】

> 有一个 $n \times m$ 的棋盘，在某个点 (x, y) 上有一个马，要求你计算出马到达棋盘上任意一个点最少要走几步。

【输入描述】

> 输入只有一行四个整数，分别为 n, m, x, y。

【输出描述】

> 一个 $n \times m$ 的矩阵，代表马到达某个点最少要走几步（不能到达则输出 -1）。

【示例一】

> 输入：
>
> ```
> 3 3 1 1
> ```
>
> 输出：
>
> ```
> 0  3  2
> 3  -1 1
> 2  1  4
> ```

【解法】

题目要求到达某个点最少要走几步，因此可以用 bfs 解决。因为当权值为 1 时，bfs 每次都是扩展距离起点等距离的一层，天然具有最短性。

那就从起点开始，一层一层的往外搜，用一个 $dist$ 数组记录最短距离。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <cstring>

using namespace std;

typedef pair<int, int> PII;

const int N = 410;

int n, m, x, y;
int dist[N][N];

int dx[] = {1, 2, 2, 1, -1, -2, -2, -1};
int dy[] = {2, 1, -1, -2, -2, -1, 1, 2};
```

```cpp

void bfs()
{
    memset(dist, -1, sizeof dist);

    queue<PII> q;
    q.push({x, y});
    dist[x][y] = 0;

    while(q.size())
    {
        auto t = q.front(); q.pop();
        int i = t.first, j = t.second;
        for(int k = 0; k < 8; k++)
        {
            int x = i + dx[k], y = j + dy[k];
            if(x < 1 || x > n || y < 1 || y > m) continue;
            if(dist[x][y] != -1) continue;

            dist[x][y] = dist[i][j] + 1; // 更细结果
            q.push({x, y});
        }
    }
}

int main()
{
    cin >> n >> m >> x >> y;

    bfs();

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            cout << dist[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

2.1.2 kotori和迷宫

> **题目来源：** 牛客网
>
> **题目链接：** kotori和迷宫
>
> **难度系数：** ★★

【题目描述】

> $kotori$ 在一个 $n \times m$ 迷宫里，迷宫的最外层被岩浆淹没，无法涉足，迷宫内有 k 个出口。
> $kotori$ 只能上下左右四个方向移动。她想知道有多少出口是她能到达的，最近的出口离她有多远？

【输入描述】

> 第一行为两个整数 n 和 m，代表迷宫的行和列数 $(1 \leq n, m \leq 30)$
>
> 后面紧跟着 n 行长度为 m 的字符串来描述迷宫。`'k'` 代表 $kotori$ 开始的位置，`'.'` 代表道路，`'*'` 代表墙壁，`'e'` 代表出口。保证输入合法。

【输出描述】

> 若有出口可以抵达，则输出 2 个整数，第一个代表 $kotori$ 可选择的出口的数量，第二个代表 $kotori$ 到最近的出口的步数。（注意， $kotori$ 到达出口一定会离开迷宫）
> 若没有出口可以抵达，则输出 -1。

【示例一】

> 输入：
>
> 6 8
>
> e.*.e.
>
> .**.*.e
>
> ..*k**..
>
> ***.**.e
>
> .*.*.
>
> *.....e
>
> 输出：
>
> 2 7

【解法】

经典 bfs 问题。

从迷宫的起点位置逐层开始搜索，每搜到一个点就标记一下最短距离。当把整个迷宫全部搜索完毕之后，扫描整个标记数组，求出出口的数量以及最短的距离。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <cstring>

using namespace std;

typedef pair<int, int> PII;

const int N = 35;

int n, m, x, y;
char a[N][N];
int dist[N][N];

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void bfs()
{
    memset(dist, -1, sizeof dist);
    queue<PII> q;
    q.push({x, y});
    dist[x][y] = 0;

    while(q.size())
    {
        auto t = q.front(); q.pop();
        int i = t.first, j = t.second;
        for(int k = 0; k < 4; k++)
        {
            int x = i + dx[k], y = j + dy[k];
            if(x >= 1 && x <= n && y >= 1 && y <= m && a[x][y] != '*' &&
dist[x][y] == -1)
            {
                dist[x][y] = dist[i][j] + 1;

                if(a[x][y] == 'e')
                {
                    continue;
                }
                q.push({x, y});
            }
        }
    }
}

int main()
```

```
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            cin >> a[i][j];
            if(a[i][j] == 'k')
            {
                x = i; y = j;
            }
        }
    }

    bfs();

    // 统计结果
    int cnt = 0, ret = 1e9;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            if(a[i][j] == 'e' && dist[i][j] != -1)
            {
                cnt++;
                ret = min(ret, dist[i][j]);
            }
        }
    }

    if(cnt == 0) cout << -1 << endl;
    else cout << cnt << " " << ret << endl;

    return 0;
}
```

2.1.3 Catch That Cow

> **题目来源：** 洛谷
>
> **题目链接：** [USACO07OPEN] Catch That Cow S
>
> **难度系数：** ★★

【题目描述】

> FJ 丢失了他的一头牛，他决定追回他的牛。已知 FJ 和牛在一条直线上，初始位置分别为 x 和 y，假定牛在原地不动。FJ 的行走方式很特别：他每一次可以前进一步、后退一步或者直接走到 $2 \times x$ 的位置。计算他至少需要几步追上他的牛。

【输入描述】

> 第一行为一个整数 $t(1 \leq t \leq 10)$，表示数据组数；
>
> 接下来每行包含一个两个正整数 $x, y(0 < x, y \leq 10^5)$，分别表示 FJ 和牛的坐标。

【输出描述】

> 对于每组数据，输出最少步数。

【示例一】

> 输入：
>
> 1
>
> 5 17
>
> 输出：
>
> 4

【解法】

可以暴力枚举出所有的行走路径，因为是求最少步数，所以可以用 bfs 解决：

- 从起点位置开始搜索，每次向外扩展三种行走方式；
- 当第一次搜到牛的位置时，就是最短距离。

如果不做任何处理，时间和空间都会超。因为我们会搜索到很多无效的位置，所以我们要加上剪枝策略：

1. 当 -1 减到负数的时候，剪掉；

   因为如果走到负数位置，还是需要回头走到正数位置，一定不是最优解。

2. 当 $+1$ 操作越过 y 的时候，剪掉；

   如果 $+1$ 之后大于 y，说明本身就在 y 位置或者 y 的右侧，你再往右走还是需要再向左走回去。一定不是最优解，剪掉。

3. 当 y 是偶数，并且当 $\times 2$ 操作之后大于 y 的时候，剪掉，因为不如先减到 y 的一半然后再乘；

   设当前数是 x，那么：

   - 先乘后减，总的步数 $t_1 = 2x - y + 1$；
   - 先减后乘，总的步数 $t_2 = x - y/2 + 1$；
   - $t_1 - t_2 = 2x - y + 1 - (x - y/2 + 1) = x - y/2 > 0$；
   - 因此，先乘后减不如先减后乘。

4. 设 y 是奇数的时候，那么 $y+1$ 就是偶数，根据 3 可得， $\times 2$ 操作不能超过 $y+1$。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <cstring>

using namespace std;

const int N = 1e5 + 10;

int n = 1e5;
int x, y;
int dist[N];

void bfs()
{
    queue<int> q;
    q.push(x);
    dist[x] = 0;

    while(q.size())
    {
        auto t = q.front(); q.pop();
        int a = t + 1, b = t - 1, c = t * 2;

        if(a <= n && dist[a] == -1)
        {
            dist[a] = dist[t] + 1;
            q.push(a);
        }

        if(b > 0 && dist[b] == -1)
        {
            dist[b] = dist[t] + 1;
            q.push(b);
        }

        if(c <= n && dist[c] == -1)
        {
            dist[c] = dist[t] + 1;
            q.push(c);
        }

```

```
42            // 剪枝
43            if(a == y || b == y || c == y) return;
44        }
45    }
46
47    int main()
48    {
49        int T; cin >> T;
50
51        while(T--)
52        {
53            // 注意清空数据
54            memset(dist, -1, sizeof dist);
55            cin >> x >> y;
56            bfs();
57            cout << dist[y] << endl;
58        }
59
60
61        return 0;
62    }
```

2.1.4 八数码难题

> **题目来源：** 洛谷
>
> **题目链接：** P1379 八数码难题
>
> **难度系数：** ★★★

【题目描述】

> 在 3×3 的棋盘上，摆有八个棋子，每个棋子上标有 1 至 8 的某一数字。棋盘中留有一个空格，空格用 0 来表示。空格周围的棋子可以移到空格中。要求解的问题是：给出一种初始布局（初始状态）和目标布局（为了使题目简单,设目标状态为 123804765），找到一种最少步骤的移动方法，实现从初始布局到目标布局的转变。

【输入描述】

> 输入初始状态，一行九个数字，空格用 0 表示。

【输出描述】

> 只有一行，该行只有一个数字，表示从初始状态到目标状态需要的最少移动次数。保证测试数据中无特殊无法到达目标状态数据。

【示例一】

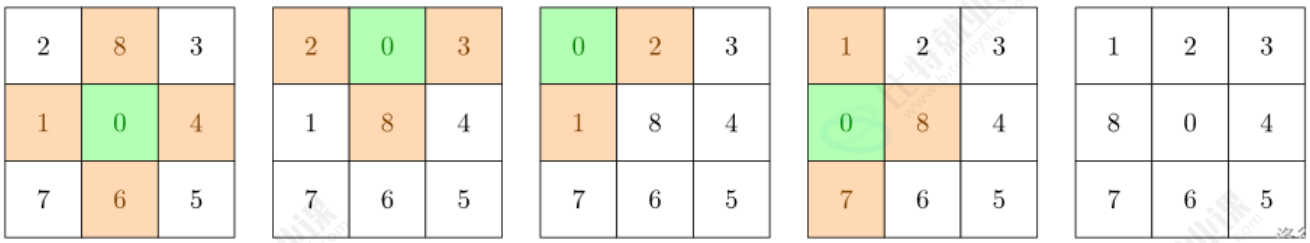
输入：

283104765

输出：

4

样例解释：



图中标有 0 的是空格。绿色格子是空格所在位置，橙色格子是下一步可以移动到空格的位置。如图所示，用四步可以达到目标状态。

【解法】

经过之前那么多题的铺垫，这道题的解法还是容易想到的。因为要求的是最短步数，因此可以用 bfs 解决。

- 从起始状态开始，每次扩展上下左右交换后的状态；
- 在搜索的过程中，第一次遇到最终状态就返回最短步数。

算法原理虽然容易，但是实现起来比较麻烦，我们要想办法处理下面几件事情：

1. 如何记录一个 3×3 的棋盘？

   可以用字符串。从上往下，从左往右将棋盘内的数依次存到一个字符串里，来标记棋盘的状态。

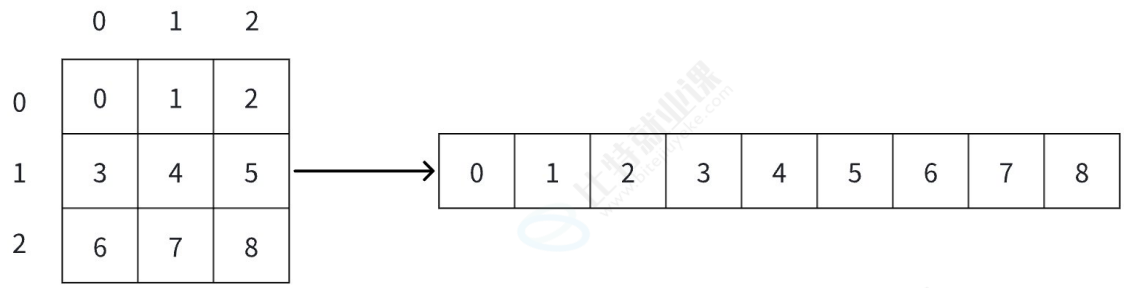
2. 如何记录最短路？

   可以用 $unordered_map < string, int >$ 来标记最短距离；

3. 如何通过一个字符串找到交换之后的字符串？

   策略一：先把字符串还原成二维矩阵，然后交换 0 与四周的数字，最后再把交换之后的棋盘还原成字符串。

   虽然可行，但是太过于麻烦。我们其实可以通过计算，快速得出二维坐标与一维下标相互转换前后的值。如下图：



1. 二维坐标(x,y) -> 一维下标(pos)： pos = x * 3 + y
   可以看成 x 行个元素，又多出来 y 个，每行 3 个，一共 3 * x + y

2. 一维下标(pos) -> 二维坐标(x,y)：x = pos / 3, y = pos % 3
   可以看成这个数能组成多少行，余数就是组成整行之后多出来的个数

这个技巧特别常用，我们可以推广到 $n \times m$ 的矩阵坐标 (x, y)，映射成一个数 pos，可以起到空间优化的效果。后续做题中我们就会遇到。

因此，我们可以直接在字符串中，找出交换前后的下标，直接交换字符串对应位置，就能得到交换之后的状态。

【参考代码】

```cpp
#include <iostream>
#include <unordered_map>
#include <queue>

using namespace std;

string s;
string aim = "123804765";
unordered_map<string, int> dist;

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void bfs()
{
    queue<string> q;
    q.push(s);
    dist[s] = 0;

    while(q.size())
    {
        string t = q.front(); q.pop();
```

```
23
24          int pos = 0;
25          while(t[pos] != '0') pos++;
26          int x = pos / 3, y = pos % 3; // 计算二维矩阵中对应的位置
27
28          for(int i = 0; i < 4; i++)
29          {
30              int a = x + dx[i], b = y + dy[i];
31              if(a >= 0 && a <= 2 && b >= 0 && b <= 2)
32              {
33                  string next = t;
34                  // (x, y) 与 (a, b) 做交换
35                  int p = 3 * a + b;
36                  swap(next[p], next[pos]);
37                  if(dist.count(next)) continue;
38
39                  dist[next] = dist[t] + 1;
40                  q.push(next);
41
42                  if(next == aim) return; // 剪枝
43              }
44          }
45      }
46  }
47
48  int main()
49  {
50      cin >> s;
51
52      bfs();
53
54      cout << dist[aim] << endl;
55
56      return 0;
57  }
```

2.2 多源 BFS

1. 单源最短路问题 vs 多源最短路问题

- 当问题中只存在一个起点时，这时的最短路问题就是单源最短路问题。
- 当问题中存在多个起点而不是单一起点时，这时的最短路问题就是多源最短路问题。

2. 多源 BFS

多源最短路问题的边权都为 1 时，此时就可以用多源 BFS 来解决。

3. 解决方式

把这些源点汇聚在一起，当成一个"超级源点"。然后从这个"超级源点"开始，处理最短路问题。落实到代码上时：

> 1. 初始化的时候，把所有的源点都加入到队列里面；
> 2. 然后正常执行 bfs 的逻辑即可。
>
> 也就是初始化的时候，比普通的 bfs 多加入几个起点。

2.2.1 矩阵距离

> **题目来源：** 牛客网
>
> **题目链接：** 矩阵距离
>
> **难度系数：** ★★

【题目描述】

> 给定一个 N 行 M 列的 01 矩阵 A，$A[i][j]$ 与 $A[k][l]$ 之间的曼哈顿距离定义为：$dist(A[i][j],A[k][l])=|i-k|+|j-l|$。
>
> 输出一个 N 行 M 列的整数矩阵 B，其中：
> $B[i][j]=min(1\le x\le N,1\le y\le M,A[x][y]=1)\{dist(A[i][j],A[x][y])\}$
>
> 即求与每个位置曼哈顿距离最近的 1，$N,M\le 1000$

【输入描述】

> 第一行两个整数 n,m。
>
> 接下来一个 n 行 m 列的 01 矩阵，数字之间没有空格。

【输出描述】

> 一个 n 行 m 列的矩阵 B，相邻两个整数之间用一个空格隔开。

【示例一】

> 输入：
>
> 3 4
>
> 0001
>
> 0011

```
0110
```

输出：

```
3210
2100
1001
```

【解法】

正难则反：

- 如果针对某一个点，直接去找最近的 1 ，我们需要对所有的 0 都来一次 bfs ，这个时间复杂度是接受不了的。
- 但是我们如果反着来想，从 1 开始向外扩展，每遍历到一个 0 就更新一下最短距离。这样仅需一次 bfs ，就可以把所有点距离 1 的最短距离更新出来。

正难则反是很重要的思想，后续还有很多题可以用到这个思想。

由于 1 的数量很多，因此可以把所有的 1 看成一个超级源点，从这个超级源点开始一层一层的向外扩展。实现起来也很简单，就是在初始化阶段把所有 1 的坐标加入到队列中，然后正常 bfs 。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <cstring>

using namespace std;

typedef pair<int, int> PII;

const int N = 1010;

int n, m;
char a[N][N];
int dist[N][N];

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void bfs()
{
    memset(dist, -1, sizeof dist);
```

```cpp

    queue<PII> q;
    // 1. 所有的起点加入到队列里面
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            if(a[i][j] == '1')
            {
                q.push({i, j});
                dist[i][j] = 0;
            }

    // 2. 正常的 bfs
    while(q.size())
    {
        auto t = q.front(); q.pop();
        int x = t.first, y = t.second;
        for(int i = 0; i < 4; i++)
        {
            int a = x + dx[i], b = y + dy[i];
            if(a >= 1 && a <= n && b >= 1 && b <= m && dist[a][b] == -1)
            {
                dist[a][b] = dist[x][y] + 1;
                q.push({a, b});
            }
        }
    }
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            cin >> a[i][j];

    bfs();

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            cout << dist[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
```

```
68  }
```

2.2.2 刺杀大使

题目来源： 洛谷

题目链接： P1902 刺杀大使

难度系数： ★★★

【题目描述】

某组织正在策划一起对某大使的刺杀行动。他们来到了使馆，准备完成此次刺杀，要进入使馆首先必须通过使馆前的防御迷阵。

迷阵由 $n \times m$ 个相同的小房间组成，每个房间与相邻四个房间之间有门可通行。在第 n 行的 m 个房间里有 m 个机关，这些机关必须全部打开才可以进入大使馆。而第 1 行的 m 个房间有 m 扇向外打开的门，是迷阵的入口。除了第 1 行和第 n 行的房间外，每个房间都被使馆的安保人员安装了激光杀伤装置，将会对进入房间的人造成一定的伤害。第 i 行第 j 列造成的伤害值为 $p_{i,j}$（第 i 行和第 n 行的 p 值全部为 0 ）。

现在某组织打算以最小伤害代价进入迷阵，打开全部机关，显然，他们可以选 择任意多的人从任意的门进入，但必须到达第 n 行的每个房间。一个士兵受到的伤害值为他到达某个机关的路径上所有房间的伤害值中的最大值，整个部队受到的伤害值为所有士兵的伤害值中的最大值。现在，这个恐怖组织掌握了迷阵的情况，他们需要提前知道怎么安排士兵的行进路线可以使得整个部队的伤害值最小。

【输入描述】

第一行有两个整数 n, m ，表示迷阵的大小。

接下来 n 行，每行 m 个数，第 i 行第 j 列的数表示 $p_{i,j}$ 。

对于 100% 的数据， $n, m \leq 1000$，$p_{i,j} \leq 1000$ 。

【输出描述】

输出一个数，表示最小伤害代价。

【示例一】

输入：

```
4 2
0 0
3 5
2 4
```

```
0 0
```

输出：

```
3
```

【解法】

直接找答案显然是不现实的，因为能走的路径实在是太多了，如果全都枚举出来时间上吃不消。但是题目要求的是最大值最小化，可以尝试用二分来优化枚举。

设最终结果是 x ，会发现一个性质：

- 当规定搜索过程中的最大值大于等于 x 时，我们一定可以从第一行走到最后一行；
- 当规定搜索过程中的最大值小于 x 时，我们一定不能走到最后一行。

因此，我们可以二分最终结果，通过 bfs 或者 dfs 来判断是否能走到最后一行。

如果用 dfs ，那就从第一行的每一列开始，全都搜索一遍。如果用 bfs ，可以看成多源 bfs 问题，直接把所有的源点加入队列中，然后正常搜索即可。

【参考代码】

```cpp
#include <iostream>
#include <cstring>
#include <queue>

using namespace std;

const int N = 1010;

int n, m;
int p[N][N];
bool st[N][N];

int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};

// 最大伤害值是 mid 的时候，能否通过防御迷阵
bool bfs(int mid)
{
    memset(st, 0, sizeof st);
    queue<pair<int, int>> q;
    // 把所有的源点加入到队列中
    for(int j = 1; j <= m; j++)
    {
```

```cpp
        q.push({1, j});
        st[1][j] = true;
    }

    while(q.size())
    {
        auto [a, b] = q.front(); q.pop();
        // 只要能到达最后一行的任意位置，就返回 true
        if(a == n) return true;

        for(int i = 0; i < 4; i++)
        {
            int x = a + dx[i], y = b + dy[i];
            if(x >= 1 && x <= n && y >= 1 && y <= m && !st[x][y])
            {
                if(p[x][y] > mid) continue; // 时间超过 mid，剪掉
                st[x][y] = true;
                q.push({x, y});
            }
        }
    }

    return false;
}

int main()
{
    cin >> n >> m;
    int l = 0, r = 0;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
        {
            cin >> p[i][j];
            r = max(r, p[i][j]);
        }

    // 二分最终结果
    while(l < r)
    {
        int mid = (l + r) >> 1;
        if(bfs(mid)) r = mid;
        else l = mid + 1;
    }

    cout << l << endl;

    return 0;
```

```
71  }
```

2.3 01 BFS

2.3.1 小明的游戏

> **题目来源：** 洛谷
>
> **题目链接：** P4554 小明的游戏
>
> **难度系数：** ★★

【题目描述】

> 小明最近喜欢玩一个游戏。给定一个 $n \times m$ 的棋盘，上面有两种格子 `#` 和 `@`。游戏的规则很简单：给定一个起始位置和一个目标位置，小明每一步能向上，下，左，右四个方向移动一格。如果移动到同一类型的格子，则费用是 0 ，否则费用是 1 。请编程计算从起始位置移动到目标位置的最小花费。

【输入描述】

> 输入文件有多组数据。
>
> 输入第一行包含两个整数 n, m ，分别表示棋盘的行数和列数。
>
> 输入接下来的 n 行，每一行有 m 个格子（使用 `#` 或者 `@` 表示）。
>
> 输入接下来一行有四个整数 x_1, y_1, x_2, y_2 ，分别为起始位置和目标位置。
>
> 当输入 n，m 均为 0 时，表示输入结束。
>
> 对于 100% 的数据满足： $1 \leq n, m \leq 500$。

【输出描述】

> 对于每组数据，输出从起始位置到目标位置的最小花费。每一组数据独占一行。

【示例一】

> 输入：
>
> ```
> 2 2
> @#
> #@
> 0 0 1 1
> 2 2
> @@
> ```

```
@#
0 1 1 0
0 0
```

输出：

```
2
0
```

【解法】

$01bfs$ 模板题。

- 如果走到相同格子，权值为 0 ，更新最短距离，然后加入到队头；
- 如果走到不同格子，权值为 1 ，更新最短距离，然后加入到队尾。

其余的搜索方式与常规 bfs 一模一样。

【参考代码】

```cpp
#include <iostream>
#include <deque>
#include <cstring>

using namespace std;

const int N = 510;

int n, m, x1, y1, x2, y2;
char g[N][N];
int dist[N][N];

int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};

int bfs()
{
    if(x1 == x2 && y1 == y2) return 0;

    memset(dist, -1, sizeof dist);
    deque<pair<int, int>> q;
    q.push_back({x1, y1});
    dist[x1][y1] = 0;

    while(q.size())
    {
```

```cpp
        auto [a, b] = q.front(); q.pop_front();

        for(int i = 0; i < 4; i++)
        {
            int x = a + dx[i], y = b + dy[i];
            if(x >= 1 && x <= n && y >= 1 && y <= m && dist[x][y] == -1)
            {
                char prev = g[a][b], cur = g[x][y];
                if(prev == cur) // 边权是 0
                {
                    q.push_front({x, y});
                    dist[x][y] = dist[a][b];
                }
                else // 边权是 1
                {
                    q.push_back({x, y});
                    dist[x][y] = dist[a][b] + 1;
                }

                if(x == x2 && y == y2) return dist[x2][y2];
            }
        }
    }

    return -1;
}

int main()
{
    while(cin >> n >> m)
    {
        if(!n && !m) break;

        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= m; j++)
                cin >> g[i][j];
        cin >> x1 >> y1 >> x2 >> y2;
        x1++, y1++, x2++, y2++;

        cout << bfs() << endl;
    }

    return 0;
}
```

2.3.2 Three States

> **题目来源：** 洛谷
>
> **题目链接：** Three States
>
> **难度系数：** ★★★★

【题目描述】

> 给你一个 $n \times m$ 的地图，`.` 是荒地，`#` 是石头（不能走），数字是国家（编号为 `1`、`2`、`3`），求最少把多少荒地修成路可以使得三个国家连通，无解输出 -1。
>
> $1 \leq n, m \leq 10^3$。

【输入描述】

> 第一行：两个整数 n, m
>
> 接下来 n 行 m 列个字符，表示地图

【输出描述】

> 一行，表示最少路

【示例一】

> 输入：
>
> 4 5
>
> 11..2
>
> #..22
>
> #.323
>
> .#333
>
> 输出：
>
> 2

【示例二】

> 输入：
>
> 1 5
>
> 1#2#3
>
> 输出：
>
> -1

【解法】

正难则反：

- 直接找出结果点是很麻烦的，需要枚举所有的点，然后每个点都要来一次 bfs，这样是会超时的。
- 可以依次从三个国家出发，用 bfs 计算出来到达所有点的最短距离。求出所有距离之后，重新遍历所有的点，分情况求出到三个国家的最短距离。

算法原理很简单，但是里面有很多细节需要注意：

1. 因为每个国家可能有很多点，并且国家的点与点之间不连通，因此我们要用多源 bfs 求某个国家到所有点的最短距离；
2. 国家与国家之间的点如果相连，它们之间的距离是 0 （无论是不是同一个国家，在我们这道题里面，它们的距离都是 0 ，因为都不需要修路）。那么我们这道题里面边的权值要么是 0 ，要么是 1 。因此需要用 $01bfs$ 来更新所有的距离；
3. 计算某一个点到三个国家的最短距离时，应该分情况讨论。设 a,b,c 分别表示三个国家到该点的最短距离：
   a. 如果该点是一个国家，最短距离为 $a+b+c$；
   b. 如果该点是一个荒地，最短距离为 $a+b+c-2$，因为我们计算最短距离的时候，荒地会被计算三次，所以要减去两次。

【参考代码】

```
#include <iostream>
#include <deque>
#include <cstring>

using namespace std;

const int N = 1e3 + 10;

int n, m;
char g[N][N];
int dist[4][N][N];

int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};

void bfs(int num)
{
    memset(dist[num], -1, sizeof dist[num]);

    // 将所有源点加入到队列中
    deque<pair<int, int>> q;
    for(int i = 1; i <= n; i++)
```

```cpp
        for(int j = 1; j <= m; j++)
        {
            if(g[i][j] - '0' == num)
            {
                q.push_back({i, j});
                dist[num][i][j] = 0;
            }
        }

    while(q.size())
    {
        auto [a, b] = q.front(); q.pop_front();

        for(int i = 0; i < 4; i++)
        {
            int x = a + dx[i], y = b + dy[i];
            if(x >= 1 && x <= n && y >= 1 && y <= m)
            {
                if(g[x][y] == '#' || dist[num][x][y] != -1) continue;

                if(g[x][y] == '.') // 边权是 1
                {
                    dist[num][x][y] = dist[num][a][b] + 1;
                    q.push_back({x, y}); // 放队尾
                }
                else // 边权是 0
                {
                    dist[num][x][y] = dist[num][a][b];
                    q.push_front({x, y}); // 放队头
                }
            }
        }
    }
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            cin >> g[i][j];

    // 更新三个国家距离其余点的最短距离
    bfs(1); bfs(2); bfs(3);

    int ret = 1e6 + 10;

```

```
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
        {
            int a = dist[1][i][j], b = dist[2][i][j], c = dist[3][i][j];
            if(a == -1 || b == -1 || c == -1) continue;
            // 更新结果
            if(g[i][j] == '.') ret = min(ret, a + b + c - 2);
            else ret = min(ret, a + b + c);
        }

    if(ret == 1e6 + 10) cout << -1 << endl;
    else cout << ret << endl;

    return 0;
}
```

3. Floodfill 问题

3.1 Lake Counting

> **题目来源：** 洛谷
>
> **题目链接：** P1596 [USACO10OCT] Lake Counting S
>
> **难度系数：** ★★

【题目描述】

> 由于近期的降雨，雨水汇集在农民约翰的田地不同的地方。我们用一个 $N \times M (1 \leq N, M \leq 100)$ 的网格图表示。每个网格中有水（`W`）或是旱地（`.`）。一个网格与其周围的八个网格相连，而一组相连的网格视为一个水坑。约翰想弄清楚他的田地已经形成了多少水坑。给出约翰田地的示意图，确定当中有多少水坑。

【输入描述】

> 输入第 1 行：两个空格隔开的整数： N 和 M。
>
> 第 2 行到第 $N+1$ 行：每行 M 个字符，每个字符是 `W` 或 `.`，它们表示网格图中的一排。字符之间没有空格。

【输出描述】

> 输出一行，表示水坑的数量。

【示例一】

> 输入：

> 10 12
>
> W........WW.
>
> .WWW.....WWW
>
> ....WW...WW.
>
> ....WW.
>
> ....W..
>
> ..W......W..
>
> .W.W.....WW.
>
> W.W.W.....W.
>
> .W.W......W.
>
> ..W.......W.
>
> 输出：
>
> 3

【解法】

遍历整个矩阵，当遇到一个没有标记过的水坑时：

- 计数；
- 然后用 bfs 或者 dfs 将整个湖全都标记一下。

整个矩阵遍历一遍，就能得出湖的数量。

【参考代码】

```cpp
#include <iostream>
#include <queue>
#include <cstring>

using namespace std;

const int N = 110;

int n, m;
char g[N][N];
bool st[N][N];

int dx[8] = {0, 0, 1, -1, 1, 1, -1, -1};
int dy[8] = {1, -1, 0, 0, 1, -1, 1, -1};

```

```cpp
void bfs(int i, int j)
{
    queue<pair<int, int>> q;
    q.push({i, j});
    st[i][j] = true;

    while(q.size())
    {
        auto [a, b] = q.front(); q.pop();
        for(int k = 0; k < 8; k++)
        {
            int x = a + dx[k], y = b + dy[k];
            if(x < 1 || x > n || y < 1 || y > m) continue;
            if(st[x][y] || g[x][y] == '.') continue;

            st[x][y] = true;
            q.push({x, y});
        }
    }
}

void dfs(int i, int j)
{
    st[i][j] = true;

    for(int k = 0; k < 8; k++)
    {
        int x = i + dx[k], y = j + dy[k];
        if(x < 1 || x > n || y < 1 || y > m) continue;
        if(st[x][y] || g[x][y] == '.') continue;

        dfs(x, y);
    }
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            cin >> g[i][j];

    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
```

```
63                 // 发现一个没有搜索过的 W
64                 if(g[i][j] == 'W' && !st[i][j])
65                 {
66                     ret++;
67                     // bfs(i, j); // 从[i, j]位置开始，把所有的水标记一下
68                     dfs(i, j);
69                 }
70             }
71         }
72 
73         cout << ret << endl;
74 
75         return 0;
76     }
```

3.2 填涂颜色

题目来源： 洛谷

题目链接： P1162 填涂颜色

难度系数： ★★

【题目描述】

由数字 0 组成的方阵中，有一任意形状的由数字 1 构成的闭合圈。现要求把闭合圈内的所有空间都填写成 2 。例如： 6×6 的方阵 $(n=6)$，涂色前和涂色后的方阵如下：

如果从某个 0 出发，只向上下左右 4 个方向移动且仅经过其他 0 的情况下，无法到达方阵的边界，就认为这个 0 **在闭合圈内**。闭合圈不一定是环形的，可以是任意形状，但保证**闭合圈内**的 0 是连通的（两两之间可以相互到达）。

```
0 0 0 0 0 0
0 0 0 1 1 1
0 1 1 0 0 1
1 1 0 0 0 1
1 0 0 1 0 1
1 1 1 1 1 1
```

```
0 0 0 0 0 0
0 0 0 1 1 1
0 1 1 2 2 1
1 1 2 2 2 1
1 2 2 1 2 1
1 1 1 1 1 1
```

【输入描述】

每组测试数据第一行一个整数 $n(1 \le n \le 30)$。

接下来 n 行，由 0 和 1 组成的 $n \times n$ 的方阵。

方阵内只有一个闭合圈，圈内至少有一个 0 。

【输出描述】

> 已经填好数字 2 的完整方阵。

【示例一】

> 输入：
>
> ```
> 6
> 000000
> 001111
> 011001
> 110001
> 100001
> 111111
> ```
>
> 输出：
>
> ```
> 000000
> 001111
> 011221
> 112221
> 122221
> 111111
> ```

【解法】

正难则反：直接找出被 1 包围的 0 是很困难的，因为很难确定当前搜索到的这个 0 是否是被包围的。但是，我们如果从边缘的 0 开始搜索，搜索到的 0 一定是没有被包围的。

因此，我们可以从边缘的 0 开始搜索，标记所有与边缘 0 相连的联通块。那么，没有被标记过的 0 就是被包围的。

小技巧：

- 我们可以把整个矩阵的外围包上一层 0 ，这样只用从 $[0,0]$ 位置开始搜即可。不用遍历第一行第一列，最后一行以及最后一列

【参考代码】

```
1  // 实现方式一：遍历第一行第一列，最后一行和最后一列。
2  #include <iostream>
```

```cpp
#include <queue>
#include <cstring>

using namespace std;

const int N = 35;

int n;
int g[N][N];
bool st[N][N];

int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};

// 标记所有相连的 0
void bfs(int i, int j)
{
    queue<pair<int, int>> q;
    q.push({i, j});
    st[i][j] = true;

    while(q.size())
    {
        auto [a, b] = q.front(); q.pop();

        for(int i = 0; i < 4; i++)
        {
            int x = a + dx[i], y = b + dy[i];
            if(x < 1 || x > n || y < 1 || y > n) continue;
            if(st[x][y] || g[x][y] == 1) continue;

            st[x][y] = true;
            q.push({x, y});
        }
    }
}

int main()
{
    while(cin >> n)
    {
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
                cin >> g[i][j];

        memset(st, 0, sizeof st);
        // 把没有被包围的 0 标记一下
```

```
        for(int j = 1; j <= n; j++)
        {
            if(!g[1][j] && !st[1][j]) bfs(1, j); // 第一行
            if(!g[n][j] && !st[n][j]) bfs(n, j); // 第 n 行
        }
        for(int i = 1; i <= n; i++)
        {
            if(!g[i][1] && !st[i][1]) bfs(i, 1); // 第一列
            if(!g[i][n] && !st[i][n]) bfs(i, n); // 最后一列
        }

        // 标记过的就是 0，没有标记过的就是 2
        for(int i = 1; i <= n; i++)
        {
            for(int j = 1; j <= n; j++)
            {
                if(g[i][j]) cout << g[i][j] << " ";
                else if(st[i][j]) cout << 0 << " ";
                else cout << 2 << " ";
            }
            cout << endl;
        }
    }

    return 0;
}

// 实现方式二：原始矩阵周围加上一圈 0
#include <iostream>
#include <queue>

using namespace std;

const int N = 35;

int n;
int g[N][N];
bool st[N][N];

int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};

void bfs(int i, int j)
{
    queue<pair<int, int>> q;
    q.push({i, j});
    st[i][j] = true;
```

```cpp

    while(q.size())
    {
        auto t = q.front();
        q.pop();
        int a = t.first, b = t.second;
        for(int k = 0; k < 4; k++)
        {
            int x = a + dx[k], y = b + dy[k];
            // 注意判断条件
            if(x >= 0 && x <= n + 1 && y >= 0 && y <= n + 1 && !g[x][y] && !st[x][y])
            {
                q.push({x, y});
                st[x][y] = true;
            }
        }
    }
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            cin >> g[i][j];

    // 直接从 [0, 0] 开始，就能把边缘的 0 全部标记上
    bfs(0, 0);

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            if(g[i][j]) cout << g[i][j] << " ";
            else if(st[i][j]) cout << 0 << " ";
            else cout << 2 << " ";
        }
        cout << endl;
    }

    return 0;
}
```